

本小节内容

函数调用汇编实战解析

函数调用汇编实战解析

这一小节我们针对 C 语言的指针的间接访问，函数调用，返回值，对应的汇编进行解析，首先我们编写下面 C 代码：

```
#include <stdio.h>
```

```
int add(int a, int b) {
```

```
    int ret;
```

```
    ret = a + b;
```

```
    return ret;
```

```
}
```

```
int main() {
```

```
    int a, b, ret;
```

```
    int *p;
```

```
    a = 5;
```

```
    p = &a;
```

```
    b = *p + 2;
```

```
    ret = add(a, b);
```

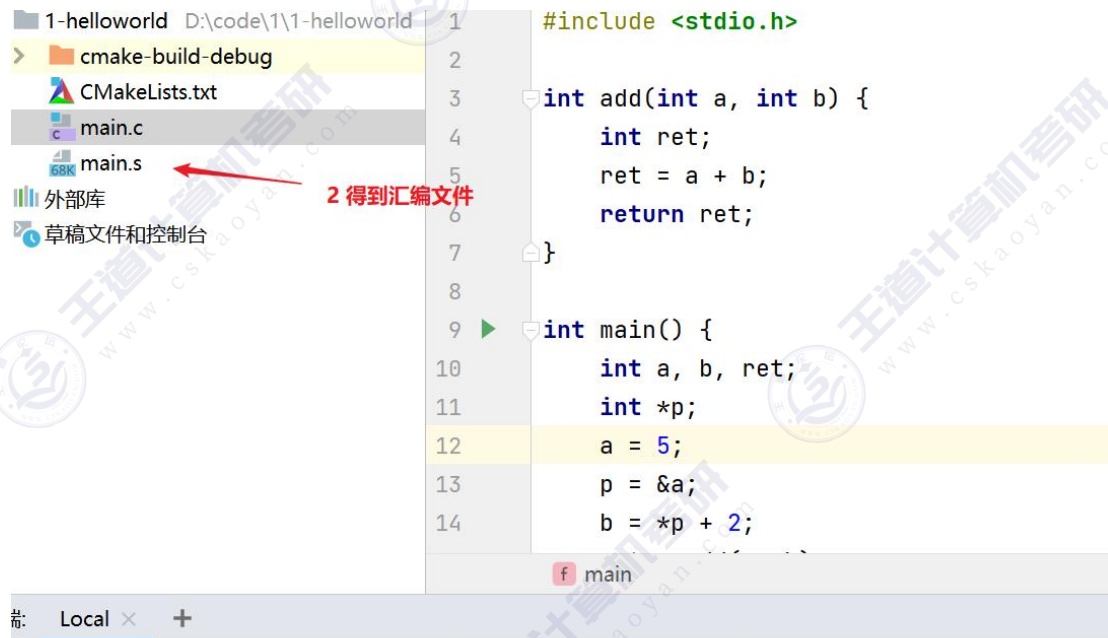
```
    printf("add result=%d\n", ret);
```



```
return 0;  
  
}
```

然后根据之前小节生成汇编的方法，在终端窗口输入

gcc -m32 -masm=intel -S -fverbose-asm main.c，如下图所示，得到汇编文件 main.s



```
(code\1\1-helloworld>gcc -m32 -masm=intel -S -fverbose-asm main.c
```

```
(code\1\1-helloworld>
```



接下来我们来分析转换后的汇编代码，首先#号代表注释，我们从 main 标签位置开看即可。我们的 C 代码在让 CPU 去运行时，其实所有的变量名都已经消失了，实际是数据从一个空间，拿到另一个空间的过程。

栈内变量先定义在低地址还是高地址，取决于操作系统与 CPU 的组合，你的可能和我的不一样，因此不用去研究，没有意义（也不属于考研大纲范围）。

我们访问所有变量的空间都是通过栈指针（esp 时刻都存着栈指针，也可以称为栈顶指针）的偏移，来获取对应变量内存空间的数据的。

函数调用的汇编原理解析

先必须明确的一点是，函数栈是向下生长的。所谓向下生长，是指从内存高地址向低地址的路径延伸。于是，栈就有栈底和栈顶，栈顶的地址要比栈底的低。

对 x86 体系的 CPU 而言，寄存器 ebp 可称为帧指针或基址指针（base pointer），寄存器 esp 可称为栈指针（stack pointer）。

这里需要说明的几点如下。

(1) ebp 在未改变之前始终指向栈帧的开始（也就是栈底），所以 ebp 的用途是在堆栈中寻址（寻址的作用会在下面详细介绍）。



(2) esp 会随着数据的入栈和出栈而移动，即 esp 始终指向栈顶。

如下图 2 所示，假设函数 A 调用函数 B，称函数 A 为调用者，称函数 B 为被调用者，则函数调用过程可以描述如下：

(1) 首先将调用者 (A) 的堆栈的基址 (ebp) 入栈，以保存之前任务的信息。

(2) 然后将调用者 (A) 的栈顶指针 (esp) 的值赋给 ebp，作为新的基址 (即被调用者 B 的栈底)。原有函数的栈顶，是新函数的栈底。

(3) 再后在这个基址 (被调用者 B 的栈底) 上开辟 (一般用 sub 指令) 相应的空间用作被调用者 B 的栈空间。

(4) 函数 B 返回后，当前栈帧的 ebp 恢复为调用者 A 的栈顶 (esp)，使栈顶恢复函数 B 被调用前的位置；然后调用者 A 从恢复后的栈顶弹出之前的 ebp 值 (因为这个值在函数调用前一步被压入堆栈)。

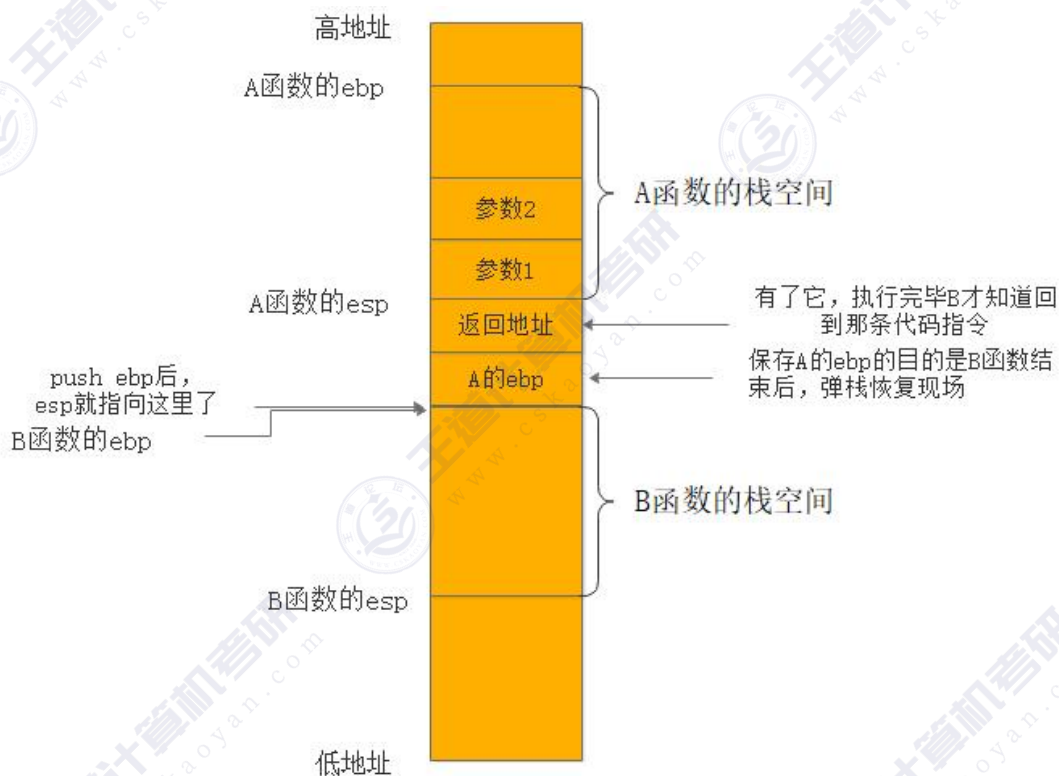


图 2 函数调用过程

这样，ebp 和 esp 就都恢复了调用函数 B 前的位置，即栈恢复函数 B 调用前的状态。相当于(ret 指令做了什么)

```
mov esp,ebp    //把 ebp 内的内容复制到 esp 寄存器中，也就是 B 函数的栈基作为原有调用者 A 函数的栈顶
pop ebp        //弹出栈顶元素，放到 ebp 寄存器中，因为原有 A 函数的栈基指针压到了内存里，所以弹出后，放入 ebp，这样原函数 A 的现场恢复完毕
```

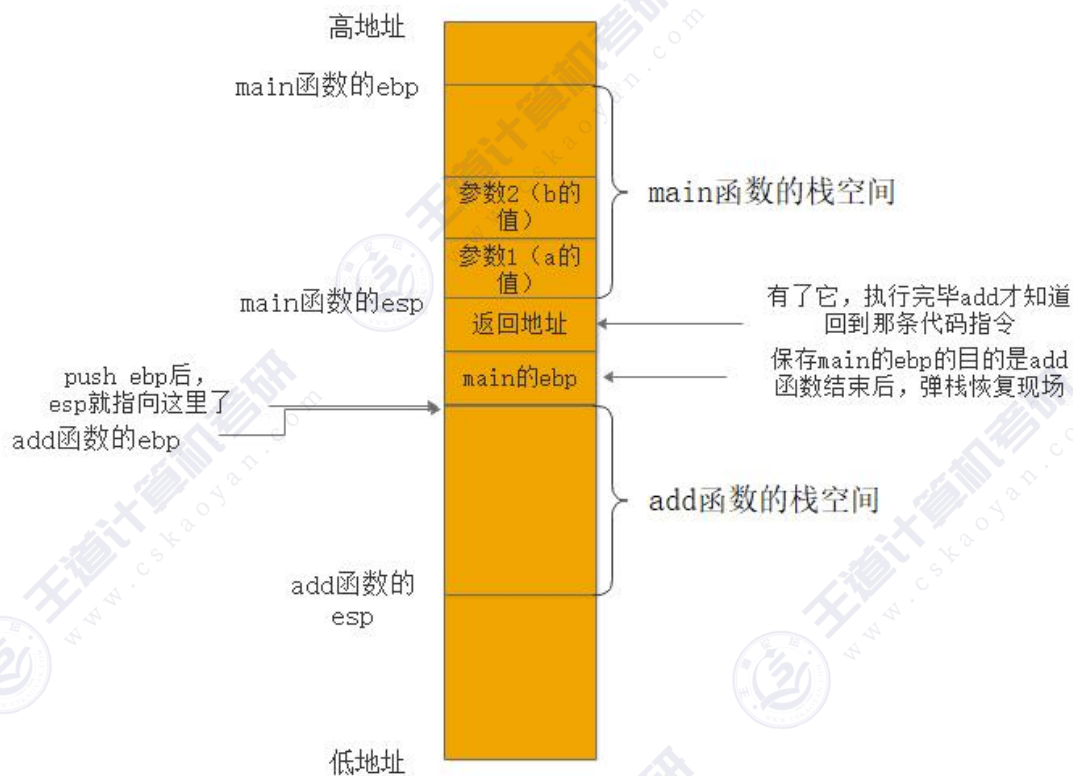


图 3 main 调用 add 的汇编原理图

```
.text
```

```
.globl _add
```

```
.def _add; .scl 2; .type 32; .endef
```

`_add`:#add 函数的入口, 这里阅读需要结合咱们的函数调用图 (图 3)

`push ebp` # 把原有函数, 也就是 main 函数的栈基指针压栈, 压栈是把 ebp 的值保存到内存上, 位置就是 esp 指向的位置

`mov ebp, esp` #, ##把 main 的栈顶指针 esp, 作为 add 函数的栈基指针 ebp

`sub esp, 16` #, #由于 add 函数自身要使用栈空间, 把 esp 减去 16, 是指 add 函数的函数栈空间大小是 16 个字节

```
# main.c:5: ret = a + b;
```

`mov edx, DWORD PTR [ebp+8]` # tmp93, a #拿到实参, 也就是 a 的值, 放入

edx

`mov eax, DWORD PTR [ebp+12] # tmp94, b#拿到实参，也就是 b 的值，放入 eax`

`add eax, edx # tmp92, tmp93 #将 eax 和 edx 相加`

`mov DWORD PTR [ebp-4], eax # ret, tmp92 #把 eax，也就是 ret 的值，放入 ebp`

减 4 个字节位置

`# main.c:6: return ret;`

`mov eax, DWORD PTR [ebp-4] # _4, ret`

`# main.c:7: }`

`leave`

`ret #函数返回,弹出压栈的指令返回地址，回到 main 函数执行`

`.def __main; .scl 2; .type 32; .endif`

`.section .rdata,"dr"`

LC0:

`.ascii "add result=%d\12\0"`

`.text`

`.globl _main`

`.def _main; .scl 2; .type 32; .endif`

`_main:`

`push ebp #`

`mov ebp, esp #,`

`and esp, -16 #,`

`sub esp, 32 #,`

`# main.c:9: int main() {`



`call` `__main` #从 main 到这里这一段我们函数调用时原理, 与 `add` 一致

main.c:12: `a = 5;`

`mov DWORD PTR [esp+16], 5` # `a`, #把常量 5 放入栈指针(`esp` 寄存器存的栈指针)偏移 16 个字节的位置, 这个位置是变量 `a` 的空间

main.c:13: `p = &a;`

`lea eax, [esp+16]` # `tmp91`, #这里用的 `lea`, 和 `mov` 不一样, 是将 `esp+16` 位置的地址, 放到 `eax` 寄存器中

`mov DWORD PTR [esp+28], eax` # `p`, `tmp91` #把 `eax` 中的值放到栈指针偏移 28 字节位置, 也就是指针变量 `p` 中

main.c:14: `b = *p + 2;` #下面两个 `mov` 是间接访问的经典解析

`mov eax, DWORD PTR [esp+28]` # `tmp92`, `p` #栈指针偏移 28 字节位置, 也就是指针变量 `p` 的值, 放到 `eax` 寄存器

`mov eax, DWORD PTR [eax]` # `_1`, `*p_5` #把 `eax` 寄存器中的值作为地址, 去内存访问到对应的数据, 放入 `eax` 中

main.c:14: `b = *p + 2;`

`add eax, 2` # `tmp93`, #对 `eax` 中的值加 2, 结果还是在 `eax` 中

`mov DWORD PTR [esp+24], eax` # `b`, `tmp93` #把 `eax` 中的值放到栈指针偏移 24 字节位置, 也就是变量 `b` 中

main.c:15: `ret = add(a, b);` #下面是函数调用, 实参传递的经典动作, 从而理解值传递是什么实现的

`mov eax, DWORD PTR [esp+16]` # `a.0_2`, `a` #栈指针偏移 16 字节位置, 也就是变量 `a` 的值, 放到 `eax` 寄存器

`mov edx, DWORD PTR [esp+24]` # `tmp94`, `b` #栈指针偏移 24 字节位置, 也就是变量 `b` 的值, 放到 `edx` 寄存器

`mov DWORD PTR [esp+4], edx` #, `tmp94` #把 `edx` 中的值 (变量 `b`), 放到寄存器栈指针偏移 4 自己的内存位置



```
mov DWORD PTR [esp], eax    #, a.0_2#把 eax 中的值（变量 a），放到寄存器栈
```

指针指向的内存位置

```
call    _add    # #调用 add 函数
```

```
mov DWORD PTR [esp+20], eax # ret, tmp95
```

```
# main.c:16:    printf("add result=%d\n", ret);
```

```
mov eax, DWORD PTR [esp+20] # tmp96, ret
```

```
mov DWORD PTR [esp+4], eax    #, tmp96
```

```
mov DWORD PTR [esp], OFFSET FLAT:LC0    #,
```

```
call    _printf    #
```

```
# main.c:17:    return 0;
```

```
mov eax, 0    # _10,
```

```
# main.c:18: }
```

```
leave
```

```
ret
```

```
.ident "GCC: (x86_64-posix-sjlj-rev0, Built by MinGW-W64 project) 8.1.0"
```

```
.def    _printf;    .scl    2; .type    32; .endef
```

这一节大家理解指针变量的间接访问原理，函数调用的原理，到这里大家对于 C 语言的每一部分在机器上的运行已经非常清晰。这一节主要掌握的指令是 **add,sub,call,ret** 等。

下面还需要掌握函数调用时，机器码的偏移值，我们前面转的都只有汇编，没有含有机码，如何得到机器码，需要执行下面两条指令。

第一条 `gcc -m32 -g -o main main.c` (Mac 一致)

第二条 `objdump --source main.exe >main.dump`(Mac 去掉.exe 后缀，写为 main 即可)

王道论坛网址: cskaoyan.com



```
401526: 55                push    %ebp
401527: 89 e5             mov     %esp,%ebp
401529: 83 e4 f0          and     $0xffffffff0,%esp
40152c: 83 ec 20          sub     $0x20,%esp
40152f: e8 ec 00 00 00    call   401620 <__main>
401534: c7 44 24 10 05 00 00 movl    $0x5,0x10(%esp)
40153b: 00
40153c: 8d 44 24 10       lea     0x10(%esp),%eax
401540: 89 44 24 1c       mov     %eax,0x1c(%esp)
401544: 8b 44 24 1c       mov     0x1c(%esp),%eax
401548: 8b 00             mov     (%eax),%eax
40154a: 83 c0 02          add     $0x2,%eax
40154d: 89 44 24 18       mov     %eax,0x18(%esp)
401551: 8b 44 24 10       mov     0x10(%esp),%eax
401555: 8b 54 24 18       mov     0x18(%esp),%edx
401559: 89 54 24 04       mov     %edx,0x4(%esp)
40155d: 89 04 24          mov     %eax,(%esp)
401560: e8 ab ff ff ff    call   401510 <_add> //掌握这里即可
401565: 89 44 24 14       mov     %eax,0x14(%esp)
401569: 8b 44 24 14       mov     0x14(%esp),%eax
40156d: 89 44 24 04       mov     %eax,0x4(%esp)
401571: c7 04 24 00 40 40 00 movl    $0x404000,(%esp)
```



```
401578: e8 bf 0f 00 00    call    40253c <_printf>
40157d: b8 00 00 00 00    mov     $0x0,%eax
401582: c9                leave
401583: c3                ret
401584: 66 90             xchg    %ax,%ax
401586: 66 90             xchg    %ax,%ax
401588: 66 90             xchg    %ax,%ax
40158a: 66 90             xchg    %ax,%ax
40158c: 66 90             xchg    %ax,%ax
40158e: 66 90             xchg    %ax,%ax
```

下面是 C，汇编，机器码三者都有。

```
int add(int a, int b) {
```

```
    40150f: 90
```

```
    nop
```

```
00401510 <_add>:
```

```
    401510: 55
```

```
    push    %ebp
```

```
    401511: 89 e5
```

```
    mov     %esp,%ebp
```

```
    401513: 83 ec 10
```

```
    sub     $0x10,%esp
```

```
    int ret;
```

```
    ret = a + b;
```

```
    401516: 8b 55 08
```

```
    mov     0x8(%ebp),%edx
```

```
    401519: 8b 45 0c
```

```
    mov     0xc(%ebp),%eax
```



```
40151c: 01 d0          add    %edx,%eax

40151e: 89 45 fc       mov    %eax,-0x4(%ebp)

    return ret;

401521: 8b 45 fc       mov    -0x4(%ebp),%eax
}

401524: c9            leave

401525: c3            ret

00401526 <_main>:

//指针的间接访问, 函数调用, 返回值

int main() {

    401526: 55            push   %ebp

    401527: 89 e5         mov    %esp,%ebp

    401529: 83 e4 f0      and    $0xffffffff,%esp

    40152c: 83 ec 20      sub    $0x20,%esp

    40152f: e8 ec 00 00 00 call   401620 <__main>

    int a, b, ret;

    int *p;

    a = 5;

    401534: c7 44 24 10 05 00 00 movl   $0x5,0x10(%esp)

    40153b: 00

    p = &a;

    40153c: 8d 44 24 10    lea    0x10(%esp),%eax
```



401540: 89 44 24 1c mov %eax,0x1c(%esp)

b = *p + 2;

401544: 8b 44 24 1c mov 0x1c(%esp),%eax

401548: 8b 00 mov (%eax),%eax

40154a: 83 c0 02 add \$0x2,%eax

40154d: 89 44 24 18 mov %eax,0x18(%esp)

ret = add(a, b);

401551: 8b 44 24 10 mov 0x10(%esp),%eax

401555: 8b 54 24 18 mov 0x18(%esp),%edx

401559: 89 54 24 04 mov %edx,0x4(%esp)

40155d: 89 04 24 mov %eax,(%esp)

401560: e8 ab ff ff ff call 401510 <_add>

401565: 89 44 24 14 mov %eax,0x14(%esp)

printf("add result=%d\n", ret);

401569: 8b 44 24 14 mov 0x14(%esp),%eax

40156d: 89 44 24 04 mov %eax,0x4(%esp)

401571: c7 04 24 00 40 40 00 movl \$0x404000,(%esp)

401578: e8 bf 0f 00 00 call 40253c <_printf>

return 0;

40157d: b8 00 00 00 00 mov \$0x0,%eax

401582: c9 leave

401583: c3 ret

401584: 66 90 xchg %ax,%ax

401586: 66 90 xchg %ax,%ax

401588: 66 90 xchg %ax,%ax

40158a: 66 90 xchg %ax,%ax

40158c: 66 90 xchg %ax,%ax

40158e: 66 90 xchg %ax,%ax